

第十四章 结构和枚举



14.1 引言

14.2 结构类型的定义

14.3 类型定义typedef

14.4 在函数间传递结构变量

14.5 结构数组

14.6 枚举



- 在日常生活中，我们经常会用到一些表格。例如，下图是一张图书目录：

书名	作者	出版商	版权日期	页数	册数	价格
书名1	张三	出版商1	2008	80	90	66
书名2	李四	出版商2	2008	78	67	77
书名3	王五	出版商3	2008	90	87	80
						
						

- 为了用计算机程序实现对上述表格的管理，需要先将上述表格进行存储。如何存储？

- 根据以前的知识，可以定义7个数组来存储100本图书信息：

```
char bookName[100][30];    //存储书名
char author[100][30];    //存储作者
char publisher[100][30]; //存储出版商名
char date[100][10];    //存储版权日期
int pages[100];    //存储页数
int copies[100];    //存储册数
float price[100];    //存储价格
```

存在的问题:

- 1.任何一本图书的信息分散在内存各处,要了解一本图书的信息,必须到各个数组的相应元素中去查找,很不方便。而且因为内存不集中,所以查找效率不高。
- 2.存储结构显得比较零散,不易管理。

而图书目录可以抽象为线性表(线性表是具有相同数据类型的 $n(n \geq 0)$ 个数据元素的有限序列),每一本图书的信息是表中的一个元素。有没有可能用一个一维数组来存储图书目录,每个数组元素存储一本图书的信息?

如果可以,那一个数组元素存储了多个不同类型的数据,那数组元素的类型又该如何定义?

- C语言在产生的时候，设计者就针对此类问题，设计了一系列用户自定义数据类型。用户可以根据具体问题的需要，设计符合自己要求的新的数据类型。结构和联合就是两种用户自定义的数据类型，结构可以用来解决上述问题。

一、结构的定义

结构是用其他类型的对象构造出来的派生数据类型（注意：结构是一种数据类型）。

```
struct 结构类型标签{  
    数据类型 数据项1;  
    数据类型 数据项2;  
    .....  
    数据类型 数据项 n;  
};
```

```
#include <stdio.h>
#define MAXTITL  82
#define MAXAUTL  42
struct Book {
    char title[MAXTITL];
    char author[MAXAUTL];
    float value;
};
int main(void)
{
    struct Book library; // 定义Book类型的结构变量
```

- 用typedef 为经常出现的类型创建一个方便、易识别的类型名。
- 例：

```
typedef struct Book Mybook;  
Mybook library[100];
```
- 用typedef来命名一个结构类型时，可以省略该结构的标签。例：

```
typedef struct {  
    double x; double y;  
} Rect;
```

14.2 结构的定义

上述图书目录的例子可以定义如下形式的结构（为了简化，假设一本书只包括书名、作者名和价格）：

```
#define  MAXTITL  82  //书名最大长度
#define  MAXAUTL  42  //作者名最大长度
typedef struct {
    char title[MAXTITL];
    char author[MAXAUTL];
    float value;
} Mybook;
```

14.2 结构的定义

1. “结构名”和“数据项”的命名规则，与变量名相同。
2. 数据类型相同的数据项，既可逐个、逐行分别定义，也可合并成一行定义。

```
typedef struct {  
    char title[MAXTITL],author[MAXAUTL];  
    float value;  
} Mybook;
```

3. 结构中的数据项，可以是不同类型（和数组不同）。既可以是整形、字符型、浮点型、指针，也允许是数组或者另一个已经定义的结构。
4. 1 个数据项称为结构的 1 个成员（或分量）。同一结构中的成员不能同名，不同结构中的成员可以同名。
5. 结构既可以在函数外定义，也可以在函数内定义。但函数内定义的结构只能在本函数中使用。



```
#include <stdio.h>
#define MAXTITL  82
#define MAXAUTL  42
typedef struct {
    char title[MAXTITL];
    char author[MAXAUTL];
    float value;
} Mybook;
int main(void)
{
    Mybook library; // 定义Mybook类型的结构变量
```



```
printf("Please enter the Book title.\n");
gets(library.title);
printf("Now enter the author.\n");
gets(library.author);
printf("Now enter the value.\n");
scanf("%f", &library.value);
printf("%s by %s: $%.2f\n", library.title,
        library.author, library.value);
printf("%s: \"%s\" ($%.2f)\n", library.author,
        library.title, library.value);
printf("Done.\n");
return 0;
}
```

14.5 逐个成员访问结构变量

一. 使用结构成员运算符“.”

通过结构变量名访问结构成员：结构变量名.结构成员名

例：

```
gets(library.title);
```

```
scanf("%f", &library.value);
```

//.运算符优先级比&高

嵌套结构:

结构中的数据项, 可以是另一个已经定义的结构变量。

```
struct Date{ /*日期结构类型*/
    int year; /*年*/
    int month; /*月*/
    int day; /*日*/
};

struct Book{
    char title[MAXTITL];
    char author[MAXAUTL];
    struct Date prtdate;
    float value;
};
```



```
#include <stdio.h>
#define MAXTITL  82
#define MAXAUTL  42
typedef struct {    /*日期结构类型*/
    int year;        /*年*/
    int month;       /*月*/
    int day;         /*日*/
} Date;
typedef struct {
    char title[MAXTITL];
    char author[MAXAUTL];
    Date prtdate;
    float value;
} Mybook;
```

```
int main(void){
    Mybook library; // 定义Book类型的结构变量
    printf("Please enter the Book title.\n");
    gets(library.title); //读取结构变量的成员
    printf("Now enter the author.\n");
    gets(library.author);
    printf("Enter the publish date: year month day.\n");
    scanf("%d%d%d", &library.prtdate.year,&
    library.prtdate.month,&library.prtdate.day);
    printf("Now enter the value.\n");
    scanf("%f", &library.value);
    printf("%s by %s: $%.2f\n",library.title,
        library.author, library.value);
    printf("Done.\n");
    return 0;
```

```
}
```

14.5 访问结构变量成员

- (1) 如果某成员本身又是一个结构类型，则只能通过多级的分量运算，对最低一级的成员进行引用。

此时的引用格式扩展为：

结构变量.成员.子成员...最低一级子成员

例如， `library.prtdate.month`

- (2) 对最低一级成员，可像同类型的普通变量一样，进行相应的各种运算。
- (3) 既可引用结构变量成员的地址，也可引用结构变量的地址。例如， `&library`，
`&library.value`

- 在定义结构变量的同时可以进行初始化

```
struct Book library={  
    "The pirate and the Devious Damsel",  
    "Renee Vivotte",  
    1.95 };
```

The pirate and the Devious Damsel
Renee Vivotte
1.95

如果初始化的个数少于结构中的成员数目，剩余成员被自动化初始为0（整型、浮点型）、‘\0’（字符或者字符串型）、NULL（指针）

14.4 结构变量可互相赋值

```
struct Book bk={"The pirate and the Devious  
Damsel","Renee Vivotte", {1980,9,20},  
1.95}; //注意出版日期的初始化
```

或者

```
struct Book bk={"The pirate and the Devious  
Damsel","Renee Vivotte", 1980,9,20,1.95};
```

- 为方便编程，C语言允许结构变量之间赋值运算。

```
struct Book bk2;  
bk2 = bk;
```

把结构传递给函数的三种方式:

- 传递单个成员
- 传递整个结构
- 传递指向结构的指针

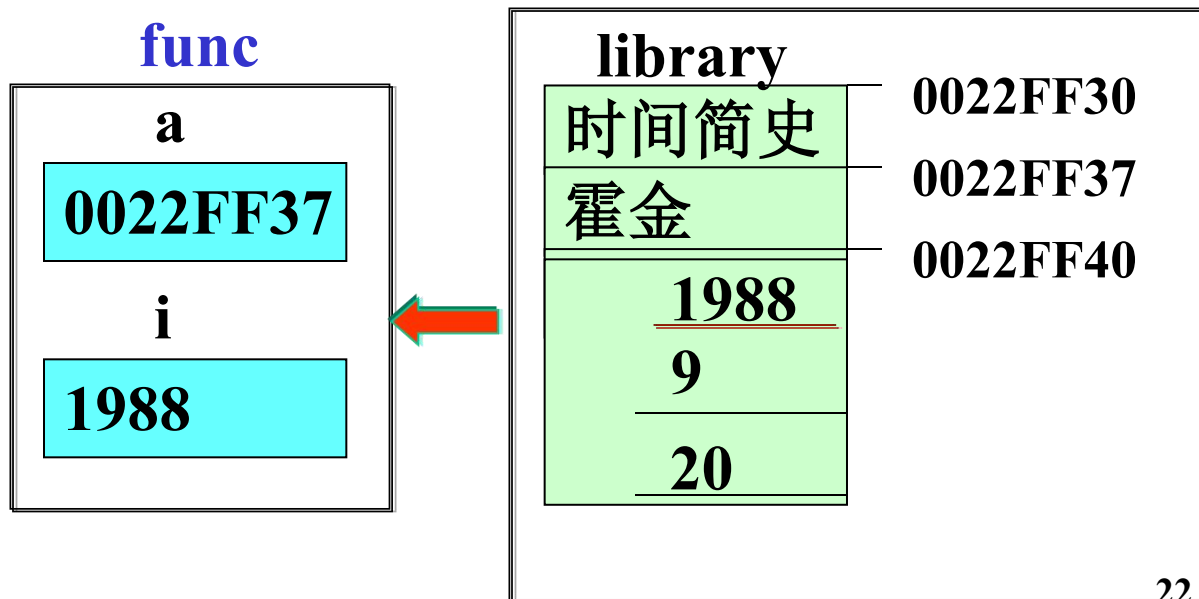
14.6 函数和结构

1. 传递单个成员(此时和传递普通变量一样)

```
void func(char a[],int i )/*函数定义*/
```

```
/*main函数中的函数调用*/
```

```
func(library.title, library.prtdate.year);
```



2. 传递整个结构（形参是实参的副本，两者互不影响）

```
void func(Mybook bk)/*函数定义*/
```

```
func(library);/* main函数中的函数调用*/
```

func

bk	
时间简史	0023FF30
霍金	0023FF37
1988	0023FF40
9	
20	

main

library	
时间简史	0022FF30
霍金	0022FF37
1988	0022FF40
9	
20	



设计打印一本图书信息的函数

```
void printbook(Mybook bk){  
    printf("%s by %s at %d-%d-%d: $%.2f\n",bk.title,  
        bk.author,  
        bk.prtdate.year,bk.prtdate.month,bk.prtdate.day,bk.value);  
  
    printf("%s: \"%s\" ($%.2f)\n", bk.author,  
        bk.title, bk.value);  
    printf("Done.\n");  
}
```

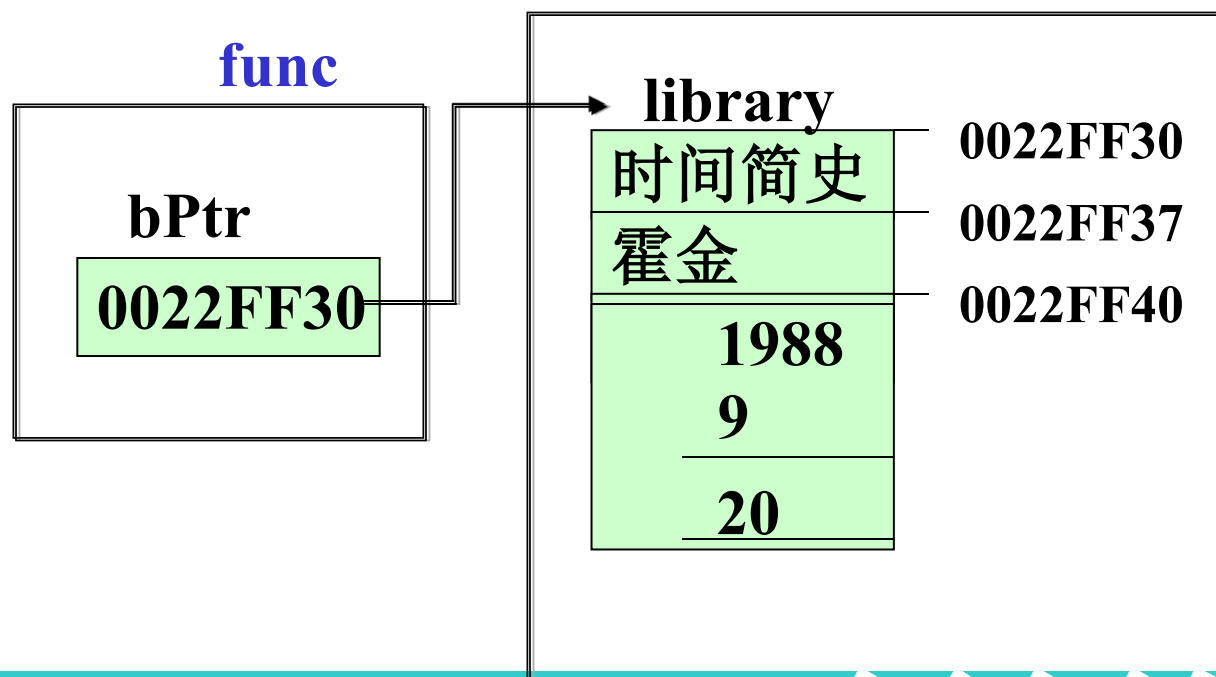
缺点：形参会复制实参的结构数据，浪费空间。

3. 传递指向结构的指针（被调用函数可以操作主调函数的结构变量）

```
void func(Mybook * pbk)/*函数定义*/
```

```
func(&library);/*函数调用*/
```

main



通过指针访问结构变量成员

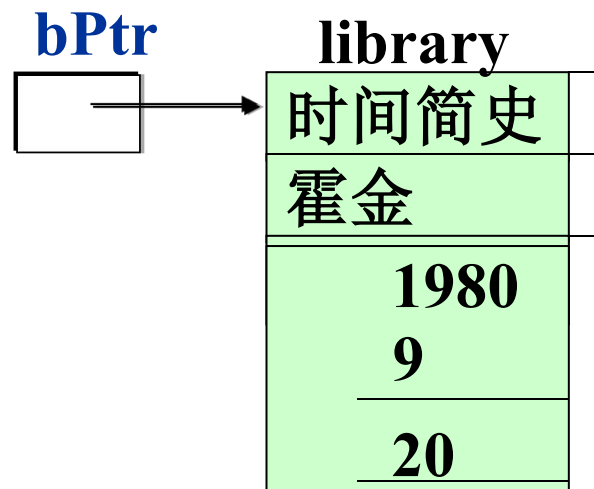
指针运算符“->”

通过指向结构变量的指针访问结构成员：

指针名->结构成员名

或 (*指针名).结构成员名

运算符. 优先级
高于运算符*,
故必须加括号





设计打印一本图书信息的函数

```
void outbook(Mybook * pbk){  
    printf("%s by %s at %d-%d-%d: $%.2f\n",pbk->title,  
        pbk->author, pbk->prtdate.year, pbk->  
        prtdate.month, pbk->prtdate.day, pbk->value);  
  
    printf("%s: \"%s\" ($%.2f)\n", pbk->author,  
        pbk->title, pbk->value);  
    printf("Done.\n");  
}
```

使用指针传递结构变量的函数示例

```
/*打印多本图书*/  
void printbooks(Mybook b[],int size){  
    for (int i=0;i<size;i++)  
        outbook(b+i); //outbook(&b[i]);  
}  
/*将图书价格打8折*/  
int updatebook(Mybook * pbk){  
    pbk->value *= 0.8;  
    //(*pbk).value = (*pbk).value * 0.8;  
}
```

使用指针的读取图书函数

```
int getbook(Mybook * pbk){  
    printf("Please enter the Book title.\n");  
    gets(pbk->title);  
    printf("Now enter the author.\n");  
    gets(pbk->author);  
    printf("Now enter the publish date: year month  
day.\n");  
    scanf("%d%d%d", &pbk->prtdate.year,&pbk->  
prtdate.month,&pbk->prtdate.day);  
    printf("Now enter the value.\n");  
    scanf("%f", &pbk->value);  
    getchar();//读取价格后面的\n  
    return 1;  
}
```



```
#include <stdio.h>
#define MAXTITL  82
#define MAXAUTL  42
typedef struct {    /*日期结构类型*/
    int year;        /*年*/
    int month;       /*月*/
    int day;         /*日*/
} Date;
typedef struct {
    char title[MAXTITL];
    char author[MAXAUTL];
    Date prtdate;
    float value;
} Mybook;
```

```
#define NUM 2
int getbook(Mybook *);
void printbooks(Mybook b[],int size);
int updatebook(Mybook *);
int main(void)
{
    Mybook library[NUM]; //定义Book类型的结构变量

    for(int i=0;i<NUM;i++) {
        getbook(library+i); //getbook(&library[i]);
        updatebook(library+i); //updatebook(&library[i]);
    }

    printbooks(library,NUM);
    return 0;
}
```

- 传递结构、还是指向结构的指针？
 - 传递指针
 - 优点：既可以工作在较早的C实现上，也可以工作在较新的C实现上；执行速度快：只需要传递一个地址。
 - 缺点：缺少对数据的保护（可以使用`const`限定词解决这个问题）。
 - 传递结构：
 - 优点：安全性。函数处理的是原始数据的副本。
 - 缺点：
 - 早期的C实现不处理这种代码
 - 浪费时间和空间

- 洗牌和发牌游戏模拟

```
#include<stdio.h>
#include<stdlib.h>
#include<time.h>
struct Card{
    char * face;
    char * suit;
};
typedef struct Card Mycard;
void fillDeck2(Mycard * wDeck,char *
wFace[],char * wSuit[]);
void shuffle(Mycard * wDeck);
void deal(Mycard * wDeck);
```



suit[0]

suit[1]

suit[2]

suit[3]

suit

H e a r t s \0

D i a m o n d s \0

C l u b s \0

S p a d e s \0

face[0]

face[11]

face[12]

face

A c e \0

...

...

Q u e e n \0

K i n g \0

```
int main()
{
    Mycard deck[52];
    const char *
    face[]={ "Ace", "Deuce", "Three", "Four", "Five",
             "Six", "Seven", "Eight", "Nine", "Ten",
             "Jeck", "Queen", "King"};

    const char *
    suit[]={ "Hearts", "Diamonds", "Clubs", "Spades"};

    srand((unsigned)time(NULL));

    fillDeck2(deck, face, suit); //初始化一副牌
    shuffle(deck); //洗牌
    deal(deck); //发牌

    return 0;
}
```

初始化一副牌（算法1）

```
void fillDeck1(MyCard * wDeck, char * wFace[], char *  
    wSuit[])  
{  
    int i=0, suit, face;  
    for(suit=0; suit<=3; suit++)  
        for(face=0; face<=12; face++){  
            wDeck[i].suit=wSuit[suit];  
            wDeck[i].face=wFace[face];  
            i++;  
        }  
}
```

初始化一副牌（算法2）

```
void fillDeck2(Mycard * wDeck,char *  
    wFace[],char * wSuit[]) //初始化牌  
{  
    int i;  
    for(i=0;i<52;i++){  
        wDeck[i].face=wFace[i%13];  
        wDeck[i].suit=wSuit[i/13];  
    }  
}
```

```
void shuffle(Mycard * wDeck) //洗牌
{
    int i, j;
    Mycard temp;

    for(i=0;i<52;i++){
        j=rand()%52;
        temp=wDeck[i];
        wDeck[i]=wDeck[j];
        wDeck[j]=temp;
    }
}
```

洗牌思路：每张牌都和随机抽取的另外一张牌交换一次。

```
void deal(Myocard * wDeck)//发牌
{
    int i;
    for(i=0;i<52;i++){
        printf("%5s of %- 8s%c",wDeck[i].face,
                wDeck[i].suit,  (i+1)%2?'\\t':'\\n' );
    }
}
```

在实际问题中，有些变量的取值被限定在一个有限的范围内。

例如，一个星期内只有七天，一年只有十二个月，一个班每周有六门课程等等。

目前为止，我们可能写出的是这样的代码：

```
#define JAN 1
#define FEB 2
.....
#define DEC 12
for(month=JAN;month<=DEC;month++)
    printf("%2d,%11s\n",month,
monthName[month]);
```

要定义12个符号常量！能否有其他方法？

枚举是C语言提供的一种用户自定义数据类型。

例：

```
enum Months{JAN,FEB,MAR,APR,MAY,JUN,  
            JUL,AUG,SEP,OCT,NOV,DEC};
```

类型名：Months

类型的取值范围：{JAN,FEB,MAR,APR,MAY,JUN,
JUL,AUG,SEP,OCT,NOV,DEC};

其中：枚举元素按常量处理，故称枚举常量。JAN值为0，
FEB值为1，MAR值为2，依次类推。

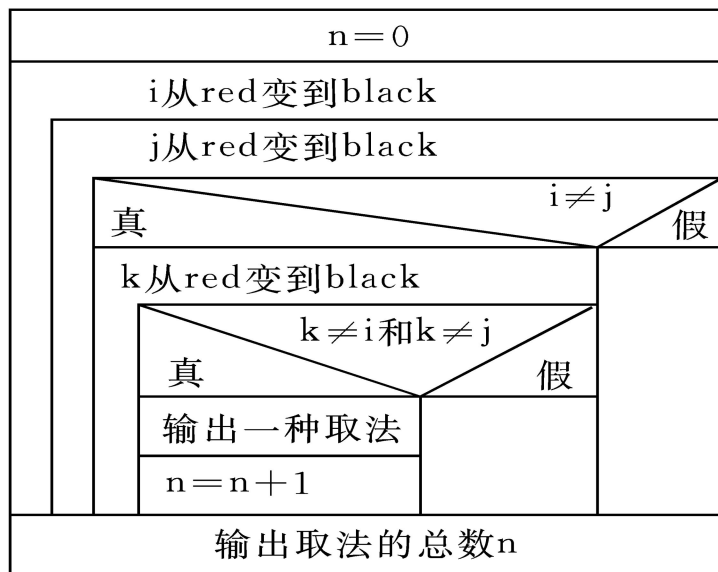
```
enum Months{JAN=1,FEB,MAR,APR,MAY,JUN,  
            JUL,AUG,SEP,OCT,NOV,DEC};  
JAN值为1，FEB值为2，MAR值为3，依次类推
```

```
enum Months{JAN=1,FEB,MAR,APR,MAY,JUN,
             JUL,AUG,SEP,OCT,NOV,DEC};

int main(){
    enum Months month;
    char *
monthName[]={ "", "January", "February", "March", "April", "May",
               "June", "July", "August", "September", "October", "November",
               "December"};
    scanf("%d",&month);
    //month= 3;//month = MAR;
    if (month>DEC || month<JAN)
        printf("input error!\n");
    else
        printf("%2d,%11s\n",month,monthName[month]);
    return 0;
}
```

枚举应用实例

例：口袋中有红、黄、蓝、白、黑5种颜色的球若干个。每次从口袋中先后取出3个球，问得到3种不同色的球的可能取法，输出每种排列的情况。



loop 由 1 到 3				
loop 的值				
1	2	3		
$i \Rightarrow \text{pri}$	$j \Rightarrow \text{pri}$	$k \Rightarrow \text{pri}$		
pri 的值				
red	yellow	blue	white	black
输出	输出	输出	输出	输出
"red"	"yellow"	"blue"	"white"	"black"

```
enum Color {red,yellow,blue,white,black};
int main(){
    int i,j,k,n=0;
    char * colStr[5]={"red","yellow","blue","white","black"};
    for (i=red;i<=black;i++)
        for (j=red;j<=black;j++)
            if (i!=j)
                { for (k=red;k<=black;k++)
                    if ((k!=i) && (k!=j))
                        {n=n+1; printf("%d:",n);
                          printf("%s,",colStr[i]);
                          printf("%s,",colStr[j]);
                          printf("%s\n",colStr[k]);
                        }
                }
    }
```

